

Project Documentation

KryptoFix

Local-first, encrypted password manager for Android

App created by Daniel Kiebler

Version 1.0.0 · October 14, 2026

Table of Contents

1. Introduction 3

1.1 What is KryptoFix? 3

1.2 Motivation and design goals 3

1.3 Intended audience 3

1.4 Scope of this release 3

2. Technical Details 4

2.1 Architecture 4

2.2 Technology stack 4

2.3 Cryptographic design 4

2.4 Vault file format (KFX1) 5

2.5 Security mechanisms 5

2.6 Password strength heuristic 6

2.7 Build process 6

3. Functional Scope 7

3.1 Screens and features 7

3.2 Feature status overview 7

3.3 Known limitations and recommendations 8

3.4 Roadmap 8

4. Complete Source Code 9

4.1 AndroidManifest.xml (29 lines) 9

4.2 res/values/strings.xml (3 lines) 9

4.3 src/de/kryptofix/passwordmanager/Vault.java (238 lines) 9

4.4 src/de/kryptofix/passwordmanager/MainActivity.java (850 lines) 12

Document information

Item	Value
Project	KryptoFix – local-first password manager for Android
Author / creator	Daniel Kiebler
Document version	1.0 (October 14, 2026)
Application version	1.0.0 (versionCode 1) – “Core Edition”
Package name	de.kryptofix.passwordmanager
Platform	Android 8.0+ (minSdk 26), targetSdk 34, compiled against API 34
Deliverable	KryptoFix-1.0.0.apk (45,542 bytes)
APK SHA-256	`7580d35bad3ce0fec9e40e2c9aae5300c3fc9064fbee700f75f130cf2720a58c`

1. Introduction

1.1 What is KryptoFix?

KryptoFix is a password manager for Android that keeps all credentials strictly on the user's own device. The user chooses a single master password; from it an encryption key is derived, and every stored login (title, user name, URL, password and notes) is saved inside one encrypted vault file in the app's private storage. There is no account, no cloud synchronisation, no advertising and no telemetry – the application does not even request the Android INTERNET permission, so it is technically unable to transmit data over the network.

1.2 Motivation and design goals

- **Local-first:** the vault never leaves the device unless the user decides otherwise.
- **Standard cryptography only:** the app relies exclusively on the platform's JCA/JCE implementations of PBKDF2-HMAC-SHA256 and AES-256-GCM. No custom cryptographic primitives are implemented.
- **Minimal attack surface:** no third-party libraries, no network access, no cloud backup, no embedded web content. The complete application consists of two Java classes.
- **Fail-safe handling of secrets:** the vault locks as soon as the app leaves the foreground, screenshots and screen recording are blocked, and copied passwords are removed from the clipboard automatically.
- **Transparency:** the full source code is part of this document (Chapter 4).

1.3 Intended audience

This document is aimed at developers, reviewers and security-minded users who want to understand exactly how KryptoFix stores and protects data, what the application can and cannot do in its current version, and how the delivered APK was produced.

1.4 Scope of this release

Version 1.0.0 is a deliberately compact *Core Edition*. It was built with the classic Android command-line toolchain instead of Gradle and uses only the Android framework (no Jetpack Compose, no AndroidX). It delivers a complete, usable vault with creation, editing, search, security analysis and settings. Features of the larger product specification – KDBX compatibility, biometric unlock and the Android Autofill service – are not part of this release and are listed in Section 3.3.

2. Technical Details

2.1 Architecture

The application is split into two classes with a clear separation of responsibilities. `MainActivity` builds and controls the user interface and talks to the Android platform (window flags, clipboard, lifecycle). `Vault` owns the decrypted entry list, performs all cryptography and persists the encrypted file. The user interface is created programmatically; there are no XML layout files, which keeps the APK very small (45.5 KB).

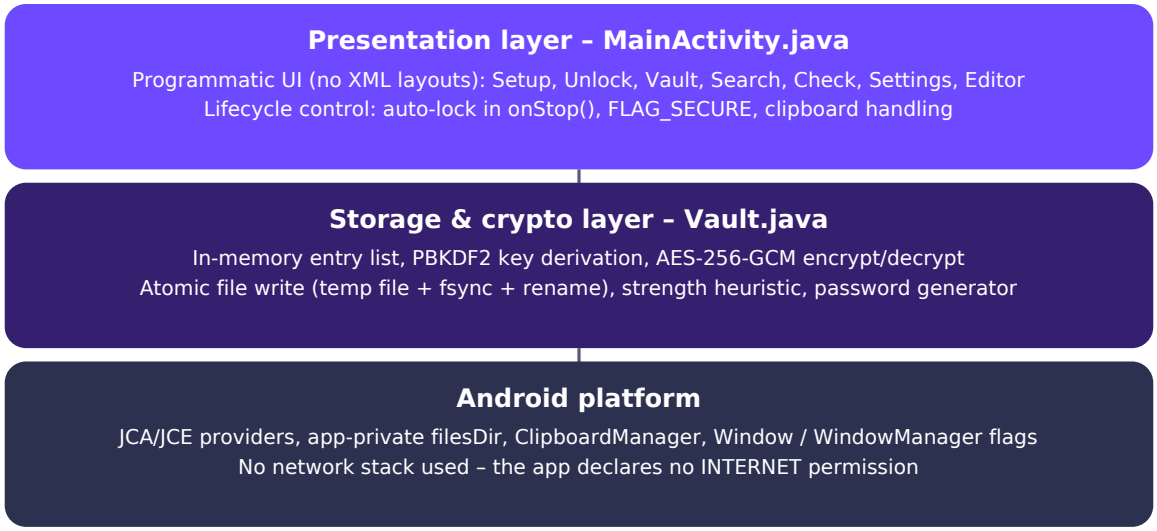


Figure 1 – Layered architecture of KryptoFix

2.2 Technology stack

Aspect	Details
Language	Java (source/target level 8), compiled with OpenJDK 21
UI framework	Android framework widgets only, built in code; Material (dark) base theme without action bar
Min / target / compile SDK	26 / 34 / 34
Dependencies	None (Android framework + JDK crypto APIs only)
Persistence	One encrypted file `filesDir/vault.kfx`; one non-sensitive SharedPreferences value (clipboard timeout)
Threading	Key derivation and (re-)keying run on a worker thread; UI updates via runOnUiThread
Distribution	Signed APK (APK Signature Scheme v2 + v3)

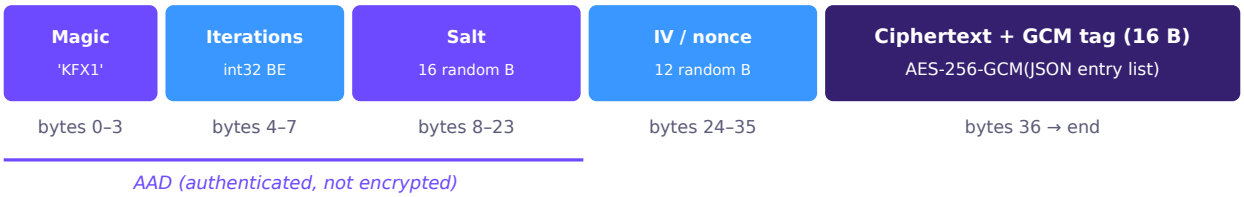
2.3 Cryptographic design

All cryptography uses standard, well-reviewed primitives provided by the platform:

Component	Choice	Rationale
Key derivation	PBKDF2WithHmacSHA256, 600,000 iterations, 16-byte random salt, 256-bit output	Slows down offline brute-force attacks; iteration count is stored in the file header so it can be raised in future versions
Encryption	AES-256 in GCM mode (AES/GCM/NoPadding), 128-bit tag	Authenticated encryption: tampering and wrong passwords are detected reliably
Nonce (IV)	12 random bytes (SecureRandom), freshly generated for **every** save	Avoids nonce reuse under the same key
Associated data	First 24 header bytes (magic, iterations, salt)	Binds the KDF parameters to the ciphertext; header manipulation makes decryption fail
Randomness	java.security.SecureRandom	Used for salts, IVs and the password generator

2.4 Vault file format (KFX1)

The vault is a single binary file. The plaintext that is encrypted is a UTF-8 JSON array containing one object per entry (`id`, `t`, `u`, `l`, `p`, `n`, `m` for id, title, user, URL, password, notes and modification time).



Writes are atomic: the new file is written to `vault.kfx.tmp`, flushed to disk with fsync and then renamed over `vault.kfx`. A crash during saving therefore cannot leave a half-written vault behind. Plaintext byte buffers are overwritten with zeros after use, and the password character arrays obtained from the input fields are zeroed after key derivation (best effort).

2.5 Security mechanisms

Mechanism	Implementation
Auto-lock	`onStop()` wipes the key, the in-memory entries and the view hierarchy whenever the app leaves the foreground (except for configuration changes). The unlock screen is shown again on return.
Screen protection	FLAG_SECURE blocks screenshots, screen recording and the task-switcher preview.
Clipboard hygiene	Copied passwords are flagged as sensitive (API 24+). After a configurable delay (15/30/60/120 s, default 30 s) the clipboard is cleared – but only if it still contains the value KryptoFix copied.
No network	The manifest declares no INTERNET permission; cleartext traffic is disabled.
No backup	`android:allowBackup="false"` excludes the vault from Android backups.
Wrong-password handling	Failed unlocks are reported generically; AES-GCM authentication guarantees that no partial plaintext is ever produced.

Mechanism	Implementation
Password generator	20 characters from four classes (lower, upper, digits, symbols), ambiguous characters excluded, at least one of each class, Fisher–Yates shuffle with SecureRandom.

2.6 Password strength heuristic

`Vault.strength()` returns a score from 0 (very weak) to 4 (strong). The score is based on length (<8, 8–11, 12–15, ≥16 characters) and on the number of character classes used; passwords using only a single character class are capped at “weak”. The result is always shown as text with a symbol (✔, ●, ⚠) in addition to colour. This is a pragmatic heuristic, not a full entropy estimator.

2.7 Build process

Because Gradle and the Google Maven repositories were not reachable in the build environment, the APK was produced with the classic command-line toolchain. The following steps reproduce the delivered artefact:

```
1. aapt package -f -m -J gen -M AndroidManifest.xml -S res -I android-34.jar \
        --min-sdk-version 26 --target-sdk-version 34 --version-code 1 \
        --version-name 1.0.0 -F res.ap_
2. javac -source 8 -target 8 -bootclasspath android-34.jar -d classes src/.../*.java
3. dx --dex --min-sdk-version=26 --output=classes.dex classes
4. cp res.ap_ unsigned.apk && aapt add unsigned.apk classes.dex
5. zipalign -f -p 4 unsigned.apk aligned.apk
6. keytool -genkeypair -keystore kryptofix-release.jks -storetype PKCS12 \
        -alias kryptofix -keyalg RSA -keysize 4096 -validity 9125 ...
7. apksigner sign --ks kryptofix-release.jks --ks-key-alias kryptofix \
        --out KryptoFix-1.0.0.apk aligned.apk
8. apksigner verify --verbose KryptoFix-1.0.0.apk     # v2 + v3: verified
```

Note on signing: the APK is signed with a self-generated key created in the build environment. That key is not a production release key. To publish updates or distribute the app, sign it with your own long-term key and keep that key safe – Android only accepts updates signed with the same key.

3. Functional Scope

3.1 Screens and features

Screen	Functionality
Setup	First start: choose and confirm a master password (min. 10 characters), live strength indicator, warning that the password cannot be recovered. Key derivation runs in the background.
Unlock	Master password entry; the vault is decrypted and verified in one step (AES-GCM). Clear error message on failure.
Vault (tab 1)	Alphabetical list of all entries with initial badge, title and user name / URL; floating “+” button to add an entry; empty-state hint.
Search (tab 2)	Live full-text search across title, user name, URL and notes (case-insensitive).
Security check (tab 3)	Local analysis: weak passwords, reused passwords, entries without password. Summary card plus tappable lists of affected entries.
Settings (tab 4)	Clipboard timeout (cycle 15/30/60/120 s), lock now, change master password (new salt, re-encryption), delete vault (with confirmation), security facts about the app.
Entry editor	Title, user name, URL, password, notes; show/hide password, generate password (20 chars), copy user name / password, live strength indicator, save, delete (with confirmation).
Change master password	Requires the vault to be unlocked; derives a new key from a fresh random salt and re-encrypts the vault.

3.2 Feature status overview

Feature	Status	Notes
Encrypted local vault (AES-256-GCM, PBKDF2)	Implemented	Format KFX1
Create / edit / delete entries	Implemented	With confirmation dialogs for destructive actions
Search	Implemented	Title, user, URL, notes
Password generator	Implemented	20 characters, SecureRandom
Strength indicator and security check	Implemented	Heuristic-based, fully offline
Auto-lock on leaving the app	Implemented	Immediate; no configurable delay yet
Clipboard auto-clear	Implemented	Requires the app process to stay alive during the timeout
Screenshot / recording protection	Implemented	FLAG_SECURE
Change master password	Implemented	
KDBX (KeePass) compatibility	Not included	Planned; requires a vetted KDBX library
Biometric unlock	Not included	Planned (Android Keystore + BiometricPrompt)
Android Autofill service	Not included	Planned
Import / export	Not included	Planned

Feature	Status	Notes
Configurable auto-lock timer	Not included	Planned

3.3 Known limitations and recommendations

- **No device testing:** the APK was verified structurally (compilation, packaging, signature, manifest inspection) but has not yet been exercised on a physical device or emulator. A functional test pass is recommended before relying on it for real credentials.
- **Memory handling:** decrypted entries are held as Java Strings while the vault is unlocked and cannot be reliably wiped from memory; the vault is locked and references are dropped as soon as the app is stopped.
- **Brute-force resistance:** protection against offline attacks relies on a strong master password and the PBKDF2 cost factor. The key is not additionally protected by hardware (Android Keystore).
- **Clipboard clearing** is performed by a timer inside the app process; if Android terminates the process before the timeout, the clipboard is not cleared.
- **No backup or sync:** uninstalling the app or clearing its data permanently deletes the vault. Users need to keep their own safe copy of important credentials until an export feature exists.
- **Signing key:** see the note in Section 2.7 – replace the test key with a production key before distribution.
- **Master password:** it cannot be recovered or reset by design.

3.4 Roadmap

Natural next steps are: (1) migrate to a Gradle/Compose project and the specified KDBX storage format using an audited library, (2) biometric unlock with a Keystore-wrapped key, (3) Autofill service, (4) encrypted export/import, (5) configurable auto-lock, and (6) an automated test suite for the vault format and the strength heuristic.

4. Complete Source Code

This chapter reproduces the complete source of the delivered application. Line numbers are added for reference; lines longer than 112 characters are wrapped and marked with the symbol ↳. The listing contains four files: the Android manifest, the string resource, `Vault.java` and `MainActivity.java`.

4.1 AndroidManifest.xml (29 lines)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="de.kryptofix.passwordmanager"
    android:versionCode="1"
    android:versionName="1.0.0">

    <uses-sdk android:minSdkVersion="26" android:targetSdkVersion="34" />
    <!-- Bewusst KEINE INTERNET-Berechtigung: local-first, keine Netzwerkzugriffe. -->

    <application
        android:label="@string/app_name"
        android:icon="@mipmap/ic_launcher"
        android:allowBackup="false"
        android:usesCleartextTraffic="false"
        android:supportsRtl="true"
        android:theme="@android:style/Theme.Material.NoActionBar">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:configChanges="orientation|screenSize|keyboardHidden|screenLayout|smallestScreenSize|keyboar
↳ d"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

4.2 res/values/strings.xml (3 lines)

```
<?xml version="1.0" encoding="utf-8"?>
<resources><string name="app_name">KryptoFix</string></resources>
```

4.3 src/de/kryptofix/passwordmanager/Vault.java (238 lines)

```
package de.kryptofix.passwordmanager;

import android.content.Context;
import org.json.JSONArray;
import org.json.JSONObject;

import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.nio.ByteBuffer;
import java.nio.charset.StandardCharsets;
import java.security.SecureRandom;
import java.util.ArrayList;
import java.util.List;

import javax.crypto.Cipher;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.PBEKeySpec;
import javax.crypto.spec.SecretKeySpec;

/**
 * Verschluesselter lokaler Tresor.
 * Primitive: ausschliesslich Standard-JCA (PBKDF2-HMAC-SHA256, AES-256-GCM) - keine Eigenentwicklung.
 * Dateiformat: "KFX1" | iterations(int) | salt(16) | iv(12) | ciphertext+tag.
 * Die ersten 24 Bytes (Magic, Iterationen, Salt) sind als AAD authentifiziert.
 */
final class Vault {
    static final class Entry {
        String id = java.util.UUID.randomUUID().toString();
        String title = "", user = "", url = "", pw = "", notes = "";
```

```
        long modified = System.currentTimeMillis();
    }

    static final int ITERATIONS = 600000;
    private static final byte[] MAGIC = {'K', 'F', 'X', '1'};
    static final List<Entry> entries = new ArrayList<>();
    private static SecretKeySpec key;
    private static byte[] salt;
    private static int iter;
    private static final SecureRandom RNG = new SecureRandom();

    private Vault() {}

    static File file(Context c) { return new File(c.getFilesDir(), "vault.kfx"); }
    static boolean exists(Context c) { return file(c).exists(); }
    static boolean unlocked() { return key != null; }

    static byte[] rand(int n) { byte[] b = new byte[n]; RNG.nextBytes(b); return b; }

    private static byte[] derive(char[] pw, byte[] s, int it) throws Exception {
        PBEKeySpec spec = new PBEKeySpec(pw, s, it, 256);
        try {
            return SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(spec).getEncoded();
        } finally {
            spec.clearPassword();
        }
    }

    static void create(Context c, char[] pw) throws Exception {
        byte[] s = rand(16);
        key = new SecretKeySpec(derive(pw, s, ITERATIONS), "AES");
        salt = s;
        iter = ITERATIONS;
        entries.clear();
        save(c);
    }

    /** Neues Hauptpasswort setzen (frischer Salt, aktuelle Iterationszahl). */
    static void rekey(Context c, char[] pw) throws Exception {
        byte[] s = rand(16);
        SecretKeySpec k = new SecretKeySpec(derive(pw, s, ITERATIONS), "AES");
        key = k;
        salt = s;
        iter = ITERATIONS;
        save(c);
    }

    static boolean unlock(Context c, char[] pw) {
        try {
            byte[] d = readAll(file(c));
            ByteBuffer bb = ByteBuffer.wrap(d);
            byte[] m = new byte[4];
            bb.get(m);
            if (!java.util.Arrays.equals(m, MAGIC)) return false;
            int it = bb.getInt();
            if (it < 10000 || it > 5000000) return false;
            byte[] s = new byte[16];
            bb.get(s);
            byte[] iv = new byte[12];
            bb.get(iv);
            byte[] ct = new byte[bb.remaining()];
            bb.get(ct);
            SecretKeySpec k = new SecretKeySpec(derive(pw, s, it), "AES");
            Cipher ci = Cipher.getInstance("AES/GCM/NoPadding");
            ci.init(Cipher.DECRYPT_MODE, k, new GCMParameterSpec(128, iv));
            ci.updateAAD(d, 0, 24);
            byte[] pt = ci.doFinal(ct);
            JSONArray arr = new JSONArray(new String(pt, StandardCharsets.UTF_8));
            java.util.Arrays.fill(pt, (byte) 0);
            List<Entry> list = new ArrayList<>();
            for (int i = 0; i < arr.length(); i++) {
                JSONObject o = arr.getJSONObject(i);
                Entry e = new Entry();
                e.id = o.optString("id", e.id);
                e.title = o.optString("t");
                e.user = o.optString("u");
                e.url = o.optString("l");
                e.pw = o.optString("p");
                e.notes = o.optString("n");
                e.modified = o.optLong("m");
                list.add(e);
            }
            entries.clear();
            entries.addAll(list);
            key = k;
            salt = s;
            iter = it;
```

```
            return true;
        } catch (Exception e) {
            return false;
        }
    }

    static void save(Context c) throws Exception {
        if (key == null) throw new IllegalStateException("locked");
        JSONArray arr = new JSONArray();
        for (Entry e : entries) {
            JSONObject o = new JSONObject();
            o.put("id", e.id);
            o.put("t", e.title);
            o.put("u", e.user);
            o.put("l", e.url);
            o.put("p", e.pw);
            o.put("n", e.notes);
            o.put("m", e.modified);
            arr.put(o);
        }
        byte[] pt = arr.toString().getBytes(StandardCharsets.UTF_8);
        byte[] header = ByteBuffer.allocate(24).put(MAGIC).putInt(iter).put(salt).array();
        byte[] iv = rand(12);
        Cipher ci = Cipher.getInstance("AES/GCM/NoPadding");
        ci.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(128, iv));
        ci.updateAAD(header);
        byte[] ct = ci.doFinal(pt);
        java.util.Arrays.fill(pt, (byte) 0);

        File tmp = new File(c.getFilesDir(), "vault.kfx.tmp");
        FileOutputStream fos = new FileOutputStream(tmp);
        try {
            fos.write(header);
            fos.write(iv);
            fos.write(ct);
            fos.getFD().sync();
        } finally {
            fos.close();
        }
        if (!tmp.renameTo(file(c))) {
            file(c).delete();
            if (!tmp.renameTo(file(c))) throw new java.io.IOException("rename failed");
        }
    }

    static void lock() {
        key = null;
        salt = null;
        entries.clear();
    }

    static void wipe(Context c) {
        lock();
        file(c).delete();
        new File(c.getFilesDir(), "vault.kfx.tmp").delete();
    }

    private static byte[] readAll(File f) throws Exception {
        FileInputStream in = new FileInputStream(f);
        try {
            byte[] b = new byte[(int) f.length()];
            int off = 0;
            while (off < b.length) {
                int r = in.read(b, off, b.length - off);
                if (r < 0) break;
                off += r;
            }
            return b;
        } finally {
            in.close();
        }
    }

    // ---- Hilfsfunktionen ----

    /** 0 = sehr schwach ... 4 = stark */
    static int strength(String p) {
        if (p == null || p.isEmpty()) return 0;
        int classes = 0;
        boolean lo = false, up = false, di = false, sy = false;
        for (char ch : p.toCharArray()) {
            if (Character.isLowerCase(ch)) lo = true;
            else if (Character.isUpperCase(ch)) up = true;
            else if (Character.isDigit(ch)) di = true;
            else sy = true;
        }
        if (lo) classes++;
```

```
206         if (up) classes++;
207         if (di) classes++;
208         if (sy) classes++;
209         int n = p.length();
210         int s;
211         if (n < 8) s = 0;
212         else if (n < 12) s = 1;
213         else if (n < 16) s = 2;
214         else s = 3;
215         if (classes >= 3 && n >= 12) s++;
216         if (classes <= 1 && s > 1) s = 1;
217         return Math.min(s, 4);
218     }
219
220     static String generate(int len) {
221         String lo = "abcdefghijkmnopqrstuvwxyz", up = "ABCDEFGHJKLMNPQRSTUVWXYZ", di = "23456789", sy = "!@#$%&*
    ↳ ?-_+=";
222         String all = lo + up + di + sy;
223         char[] out = new char[len];
224         String[] sets = {lo, up, di, sy};
225         for (int i = 0; i < len; i++) {
226             String src = i < 4 ? sets[i] : all;
227             out[i] = src.charAt(RNG.nextInt(src.length()));
228         }
229         for (int i = len - 1; i > 0; i--) {
230             int j = RNG.nextInt(i + 1);
231             char t = out[i];
232             out[i] = out[j];
233             out[j] = t;
234         }
235         return new String(out);
236     }
237 }
```

4.4 src/de/kryptofix/passwordmanager/MainActivity.java (850 lines)

```
  1 package de.kryptofix.passwordmanager;
  2
  3 import android.app.Activity;
  4 import android.app.AlertDialog;
  5 import android.content.ClipData;
  6 import android.content.ClipDescription;
  7 import android.content.ClipboardManager;
  8 import android.content.Context;
  9 import android.content.SharedPreferences;
 10 import android.graphics.Color;
 11 import android.graphics.Typeface;
 12 import android.graphics.drawable.GradientDrawable;
 13 import android.os.Build;
 14 import android.os.Bundle;
 15 import android.os.Handler;
 16 import android.os.Looper;
 17 import android.os.PersistableBundle;
 18 import android.text.Editable;
 19 import android.text.InputType;
 20 import android.text.TextWatcher;
 21 import android.view.Gravity;
 22 import android.view.View;
 23 import android.view.ViewGroup;
 24 import android.view.WindowManager;
 25 import android.widget.Button;
 26 import android.widget.EditText;
 27 import android.widget.FrameLayout;
 28 import android.widget.LinearLayout;
 29 import android.widget.ScrollView;
 30 import android.widget.TextView;
 31 import android.widget.Toast;
 32
 33 import java.util.ArrayList;
 34 import java.util.Collections;
 35 import java.util.Comparator;
 36 import java.util.HashMap;
 37 import java.util.List;
 38 import java.util.Locale;
 39 import java.util.Map;
 40
 41 public class MainActivity extends Activity {
 42     static final int BG = 0xFF0D1020, SURF = 0xFF171B30, PRI = 0xFF6D4AFF, DEEP = 0xFF35206F, LILAC = 0xFFC7B8FF
    ↳ ,
 43             BLUE = 0xFF3997FF, TXT = 0xFFF4F2FF, TXT2 = 0xFFB8B7D3, DANGER = 0xFFFF647C, OK = 0xFF47D7A0,
 44             WARN = 0xFFFFC66D, LINE = 0xFF2C3150;
 45     static final int MATCH = ViewGroup.LayoutParams.MATCH_PARENT, WRAP = ViewGroup.LayoutParams.WRAP_CONTENT;
 46
```

```
47      private static final Handler H = new Handler(Looper.getMainLooper());
48      private static Runnable clipClear;
49
50      private FrameLayout host;
51      private SharedPreferences prefs;
52      private String screen = "";
53      private int tab = 0;
54      private String searchQuery = "";
55      private Vault.Entry editing;
56
57      // ------------------------------------------------------------ Lebenszyklus
58
59      @Override
60      protected void onCreate(Bundle b) {
61          super.onCreate(b);
62          getWindow().setFlags(WindowManager.LayoutParams.FLAG_SECURE, WindowManager.LayoutParams.FLAG_SECURE);
63          getWindow().setStatusBarColor(BG);
64          getWindow().setNavigationBarColor(BG);
65          prefs = getSharedPreferences("kf", MODE_PRIVATE);
66          host = new FrameLayout(this);
67          host.setBackgroundColor(BG);
68          host.setOnApplyWindowInsetsListener(new View.OnApplyWindowInsetsListener() {
69              @Override
70              public android.view.WindowInsets onApplyWindowInsets(View v, android.view.WindowInsets i) {
71                  v.setPadding(i.getSystemWindowInsetLeft(), i.getSystemWindowInsetTop(),
72                          i.getSystemWindowInsetRight(), i.getSystemWindowInsetBottom());
73                  return i;
74              }
75          });
76          setContentView(host);
77      }
78
79      @Override
80      protected void onStart() {
81          super.onStart();
82          route();
83      }
84
85      @Override
86      protected void onStop() {
87          super.onStop();
88          if (!isChangingConfigurations() && Vault.unlocked()) {
89              Vault.lock();
90              editing = null;
91              screen = "";
92              host.removeAllViews();
93          }
94      }
95
96      @Override
97      public void onBackPressed() {
98          if (screen.equals("editor") || screen.equals("changepw")) {
99              editing = null;
100             showMain(tab);
101         } else if (screen.equals("main") && tab != 0) {
102             showMain(0);
103         } else {
104             super.onBackPressed();
105         }
106     }
107
108     private void route() {
109         if (!Vault.exists(this)) {
110             if (!screen.equals("setup")) showSetup();
111         } else if (!Vault.unlocked()) {
112             if (!screen.equals("unlock")) showUnlock();
113         } else if (screen.isEmpty()) {
114             showMain(tab);
115         }
116     }
117
118     // ------------------------------------------------------------ UI-Helfer
119
120     int dp(int v) { return (int) (v * getResources().getDisplayMetrics().density + 0.5f); }
121
122     GradientDrawable box(int color, int radiusDp, int stroke) {
123         GradientDrawable g = new GradientDrawable();
124         g.setColor(color);
125         g.setCornerRadius(dp(radiusDp));
126         if (stroke != 0) g.setStroke(dp(1), stroke);
127         return g;
128     }
129
130     TextView text(String s, int sp, int color, boolean bold) {
131         TextView t = new TextView(this);
132         t.setText(s);
133         t.setTextSize(sp);
```

```
        t.setTextColor(color);
        if (bold) t.setTypeface(Typeface.DEFAULT_BOLD);
        return t;
    }

    EditText input(String hint, boolean secret, boolean multiline) {
        EditText e = new EditText(this);
        e.setHint(hint);
        e.setHintTextColor(TXT2);
        e.setTextColor(TXT);
        e.setTextSize(16);
        e.setBackground(box(SURF, 12, LINE));
        e.setPadding(dp(14), dp(14), dp(14), dp(14));
        e.setMinHeight(dp(52));
        if (secret) {
            e.setInputType(InputType.TYPE_CLASS_TEXT | InputType.TYPE_TEXT_VARIATION_PASSWORD);
        } else if (multiline) {
            e.setInputType(InputType.TYPE_CLASS_TEXT | InputType.TYPE_TEXT_FLAG_MULTI_LINE
                    | InputType.TYPE_TEXT_FLAG_NO_SUGGESTIONS);
            e.setMinLines(3);
            e.setGravity(Gravity.TOP);
        } else {
            e.setInputType(InputType.TYPE_CLASS_TEXT | InputType.TYPE_TEXT_FLAG_NO_SUGGESTIONS);
        }
        return e;
    }

    Button btn(String label, int fill, int fg, View.OnClickListener l) {
        Button b = new Button(this);
        b.setText(label);
        b.setAllCaps(false);
        b.setTextSize(15);
        b.setTypeface(Typeface.DEFAULT_BOLD);
        b.setTextColor(fg);
        b.setStateListAnimator(null);
        b.setMinHeight(dp(52));
        b.setBackground(fill == 0 ? box(Color.TRANSPARENT, 14, LINE) : box(fill, 14, 0));
        b.setOnClickListener(l);
        return b;
    }

    Button primaryBtn(String label, View.OnClickListener l) {
        Button b = btn(label, PRI, Color.WHITE, l);
        GradientDrawable g = new GradientDrawable(GradientDrawable.Orientation.LEFT_RIGHT, new int[]{PRI, BLUE});
        g.setCornerRadius(dp(14));
        b.setBackground(g);
        return b;
    }

    LinearLayout.LayoutParams lp(int top) {
        LinearLayout.LayoutParams p = new LinearLayout.LayoutParams(MATCH, WRAP);
        p.topMargin = dp(top);
        return p;
    }

    LinearLayout column(int pad) {
        LinearLayout l = new LinearLayout(this);
        l.setOrientation(LinearLayout.VERTICAL);
        l.setPadding(dp(pad), dp(pad), dp(pad), dp(pad));
        return l;
    }

    void setScreen(String name, View v) {
        screen = name;
        host.removeAllViews();
        host.addView(v, new FrameLayout.LayoutParams(MATCH, MATCH));
    }

    View shield(int sizeDp) {
        TextView t = text("\uD83D\uDEE1", 34, Color.WHITE, false);
        t.setGravity(Gravity.CENTER);
        GradientDrawable g = new GradientDrawable(GradientDrawable.Orientation.TL_BR, new int[]{DEEP, PRI, BLUE});
        g.setShape(GradientDrawable.OVAL);
        t.setBackground(g);
        t.setLayoutParams(new LinearLayout.LayoutParams(dp(sizeDp), dp(sizeDp)));
        return t;
    }

    void toast(String s) { Toast.makeText(this, s, Toast.LENGTH_SHORT).show(); }

    char[] chars(EditText e) {
        int n = e.length();
        char[] a = new char[n];
        e.getText().getChars(0, n, a, 0);
        return a;
```

```
    }

    String strengthLabel(int s) {
        switch (s) {
            case 0: return "\u26A0 Sehr schwach";
            case 1: return "\u26A0 Schwach";
            case 2: return "\u25CF Mittel";
            case 3: return "\u2714 Gut";
            default: return "\u2714 Stark";
        }
    }

    int strengthColor(int s) { return s <= 1 ? DANGER : s == 2 ? WARN : OK; }

    // ------------------------------------------------------------ Einrichtung / Entsperren

    void showSetup() {
        ScrollView sv = new ScrollView(this);
        sv.setFillViewport(true);
        LinearLayout l = column(24);
        l.setGravity(Gravity.CENTER_HORIZONTAL);
        sv.addView(l);
        l.addView(shield(88));
        TextView t = text("KryptoFix", 28, TXT, true);
        t.setGravity(Gravity.CENTER);
        l.addView(t, lp(16));
        TextView s = text("Tresor erstellen", 16, LILAC, false);
        s.setGravity(Gravity.CENTER);
        l.addView(s, lp(2));
        TextView info = text("Lege ein Hauptpasswort fest. Es kann nicht wiederhergestellt werden \u2013 "
                + "ohne dieses Passwort sind deine Daten verloren.", 14, TXT2, false);
        info.setGravity(Gravity.CENTER);
        l.addView(info, lp(16));
        final EditText p1 = input("Hauptpasswort (mind. 10 Zeichen)", true, false);
        final EditText p2 = input("Hauptpasswort wiederholen", true, false);
        final TextView str = text("", 14, TXT2, false);
        final TextView err = text("", 14, DANGER, false);
        l.addView(p1, lp(24));
        l.addView(str, lp(6));
        l.addView(p2, lp(12));
        l.addView(err, lp(8));
        p1.addTextChangedListener(new SimpleWatcher() {
            @Override
            public void afterTextChanged(Editable e) {
                int sc = Vault.strength(e.toString());
                str.setText(e.length() == 0 ? "" : "Stärke: " + strengthLabel(sc));
                str.setTextColor(strengthColor(sc));
            }
        });
        final Button go = primaryBtn("Tresor erstellen", null);
        go.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String a = p1.getText().toString(), b = p2.getText().toString();
                if (a.length() < 10) { err.setText("\u26A0 Das Hauptpasswort muss mindestens 10 Zeichen lang sei
↳ n."); return; }
                if (!a.equals(b)) { err.setText("\u26A0 Die Passwörter stimmen nicht überein."); return; }
                err.setText("");
                go.setEnabled(false);
                go.setText("Schlüssel wird abgeleitet …");
                final char[] pw = chars(p1);
                new Thread(new Runnable() {
                    @Override
                    public void run() {
                        String problem = null;
                        try { Vault.create(MainActivity.this, pw); } catch (Exception ex) { problem = "Fehler: "
↳  + ex.getMessage(); }
                        java.util.Arrays.fill(pw, '\0');
                        final String pr = problem;
                        runOnUiThread(new Runnable() {
                            @Override
                            public void run() {
                                if (pr != null) { err.setText(pr); go.setEnabled(true); go.setText("Tresor erste
↳ llen"); }
                                else { tab = 0; showMain(0); }
                            }
                        });
                    }
                }).start();
            }
        });
        l.addView(go, lp(8));
        setScreen("setup", sv);
    }

    void showUnlock() {
        ScrollView sv = new ScrollView(this);
```

```
        sv.setFillViewport(true);
        LinearLayout l = column(24);
        l.setGravity(Gravity.CENTER);
        sv.addView(l);
        l.addView(shield(96));
        TextView t = text("KryptoFix", 28, TXT, true);
        t.setGravity(Gravity.CENTER);
        l.addView(t, lp(16));
        TextView s = text("\uD83D\uDD12 Tresor gesperrt", 16, LILAC, false);
        s.setGravity(Gravity.CENTER);
        l.addView(s, lp(2));
        final EditText pw = input("Hauptpasswort", true, false);
        final TextView err = text("", 14, DANGER, false);
        l.addView(pw, lp(28));
        l.addView(err, lp(8));
        final Button go = primaryBtn("Entsperren", null);
        View.OnClickListener unlock = new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                if (pw.length() == 0) return;
                go.setEnabled(false);
                go.setText("Wird geprüft …");
                err.setText("");
                final char[] c = chars(pw);
                new Thread(new Runnable() {
                    @Override
                    public void run() {
                        final boolean ok = Vault.unlock(MainActivity.this, c);
                        java.util.Arrays.fill(c, '\0');
                        runOnUiThread(new Runnable() {
                            @Override
                            public void run() {
                                if (ok) { showMain(tab); }
                                else {
                                    err.setText("\u26A0 Falsches Hauptpasswort oder beschädigte Tresordatei.");
                                    pw.setText("");
                                    go.setEnabled(true);
                                    go.setText("Entsperren");
                                }
                            }
                        });
                    }
                }).start();
            }
        };
        go.setOnClickListener(unlock);
        l.addView(go, lp(8));
        setScreen("unlock", sv);
    }

    // ------------------------------------------------------------ Hauptbereich

    void showMain(int which) {
        tab = which;
        editing = null;
        LinearLayout root = new LinearLayout(this);
        root.setOrientation(LinearLayout.VERTICAL);
        FrameLayout content = new FrameLayout(this);
        root.addView(content, new LinearLayout.LayoutParams(MATCH, 0, 1f));
        switch (which) {
            case 0: buildVaultTab(content); break;
            case 1: buildSearchTab(content); break;
            case 2: buildCheckTab(content); break;
            default: buildSettingsTab(content);
        }
        root.addView(navBar());
        setScreen("main", root);
    }

    View navBar() {
        LinearLayout bar = new LinearLayout(this);
        bar.setOrientation(LinearLayout.HORIZONTAL);
        bar.setBackgroundColor(SURF);
        bar.setPadding(dp(8), dp(6), dp(8), dp(6));
        String[] names = {"Tresor", "Suche", "Check", "Einstellungen"};
        String[] icons = {"\uD83D\uDD10", "\uD83D\uDD0D", "\uD83D\uDEE1", "\u2699"};
        for (int i = 0; i < 4; i++) {
            final int idx = i;
            LinearLayout item = new LinearLayout(this);
            item.setOrientation(LinearLayout.VERTICAL);
            item.setGravity(Gravity.CENTER);
            item.setPadding(0, dp(6), 0, dp(6));
            boolean sel = i == tab;
            if (sel) item.setBackground(box(DEEP, 16, 0));
            TextView ic = text(icons[i], 20, sel ? TXT : TXT2, false);
            ic.setGravity(Gravity.CENTER);
            TextView nm = text(names[i], 11, sel ? TXT : TXT2, sel);
```

```
            nm.setGravity(Gravity.CENTER);
            item.addView(ic);
            item.addView(nm);
            item.setOnClickListener(new View.OnClickListener() {
                @Override
                public void onClick(View v) { showMain(idx); }
            });
            LinearLayout.LayoutParams p = new LinearLayout.LayoutParams(0, WRAP, 1f);
            p.setMargins(dp(3), 0, dp(3), 0);
            bar.addView(item, p);
        }
        return bar;
    }

    View header(String title, String sub) {
        LinearLayout h = new LinearLayout(this);
        h.setOrientation(LinearLayout.VERTICAL);
        h.addView(text(title, 26, TXT, true));
        if (sub != null) h.addView(text(sub, 14, TXT2, false), lp(2));
        return h;
    }

    View entryCard(final Vault.Entry e) {
        LinearLayout row = new LinearLayout(this);
        row.setOrientation(LinearLayout.HORIZONTAL);
        row.setGravity(Gravity.CENTER_VERTICAL);
        row.setPadding(dp(14), dp(12), dp(14), dp(12));
        row.setBackground(box(SURF, 16, LINE));
        String label = e.title.isEmpty() ? "?" : e.title.substring(0, 1).toUpperCase(Locale.ROOT);
        TextView badge = text(label, 18, Color.WHITE, true);
        badge.setGravity(Gravity.CENTER);
        GradientDrawable g = new GradientDrawable(GradientDrawable.Orientation.TL_BR, new int[]{PRI, BLUE});
        g.setShape(GradientDrawable.OVAL);
        badge.setBackground(g);
        row.addView(badge, new LinearLayout.LayoutParams(dp(44), dp(44)));
        LinearLayout tx = new LinearLayout(this);
        tx.setOrientation(LinearLayout.VERTICAL);
        tx.addView(text(e.title.isEmpty() ? "(ohne Titel)" : e.title, 16, TXT, true));
        String sub = e.user.isEmpty() ? e.url : e.user;
        if (!sub.isEmpty()) tx.addView(text(sub, 13, TXT2, false));
        LinearLayout.LayoutParams tp = new LinearLayout.LayoutParams(0, WRAP, 1f);
        tp.leftMargin = dp(12);
        row.addView(tx, tp);
        row.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) { showEditor(e); }
        });
        return row;
    }

    List<Vault.Entry> sorted(List<Vault.Entry> in) {
        List<Vault.Entry> l = new ArrayList<>(in);
        Collections.sort(l, new Comparator<Vault.Entry>() {
            @Override
            public int compare(Vault.Entry a, Vault.Entry b) { return a.title.compareToIgnoreCase(b.title); }
        });
        return l;
    }

    void buildVaultTab(FrameLayout content) {
        ScrollView sv = new ScrollView(this);
        LinearLayout l = column(16);
        sv.addView(l);
        l.addView(header("Tresor", Vault.entries.size() + (Vault.entries.size() == 1 ? " Eintrag" : " Einträge")
                + " \u00B7 lokal verschlüsselt"));
        if (Vault.entries.isEmpty()) {
            TextView empty = text("Noch keine Einträge.\nTippe auf \u201E+\u201C, um deinen ersten Zugang zu spe
↳ ichern.",
                    15, TXT2, false);
            empty.setGravity(Gravity.CENTER);
            l.addView(empty, lp(60));
        }
        for (Vault.Entry e : sorted(Vault.entries)) l.addView(entryCard(e), lp(10));
        View spacer = new View(this);
        l.addView(spacer, new LinearLayout.LayoutParams(MATCH, dp(90)));
        content.addView(sv, new FrameLayout.LayoutParams(MATCH, MATCH));
        Button fab = primaryBtn("+", new View.OnClickListener() {
            @Override
            public void onClick(View v) { showEditor(null); }
        });
        fab.setTextSize(26);
        GradientDrawable g = new GradientDrawable(GradientDrawable.Orientation.TL_BR, new int[]{PRI, BLUE});
        g.setShape(GradientDrawable.OVAL);
        fab.setBackground(g);
        fab.setContentDescription("Neuen Eintrag hinzufügen");
        fab.setPadding(0, 0, 0, 0);
        FrameLayout.LayoutParams fp = new FrameLayout.LayoutParams(dp(60), dp(60), Gravity.BOTTOM | Gravity.END)
```

```
    ↳ ;
476         fp.setMargins(0, 0, dp(20), dp(20));
477         content.addView(fab, fp);
478     }
479
480     void buildSearchTab(FrameLayout content) {
481         LinearLayout l = column(16);
482         l.addView(header("Suche", "Titel, Benutzer, URL und Notizen"));
483         final EditText q = input("Suchen …", false, false);
484         q.setText(searchQuery);
485         l.addView(q, lp(12));
486         final LinearLayout results = new LinearLayout(this);
487         results.setOrientation(LinearLayout.VERTICAL);
488         ScrollView sv = new ScrollView(this);
489         sv.addView(results);
490         l.addView(sv, new LinearLayout.LayoutParams(MATCH, 0, 1f));
491         q.addTextChangedListener(new SimpleWatcher() {
492             @Override
493             public void afterTextChanged(Editable e) {
494                 searchQuery = e.toString();
495                 renderResults(results);
496             }
497         });
498         renderResults(results);
499         content.addView(l, new FrameLayout.LayoutParams(MATCH, MATCH));
500     }
501
502     void renderResults(LinearLayout results) {
503         results.removeAllViews();
504         String q = searchQuery.trim().toLowerCase(Locale.ROOT);
505         if (q.isEmpty()) {
506             TextView t = text("Gib einen Suchbegriff ein.", 15, TXT2, false);
507             t.setGravity(Gravity.CENTER);
508             results.addView(t, lp(40));
509             return;
510         }
511         int n = 0;
512         for (Vault.Entry e : sorted(Vault.entries)) {
513             String hay = (e.title + "\n" + e.user + "\n" + e.url + "\n" + e.notes).toLowerCase(Locale.ROOT);
514             if (hay.contains(q)) { results.addView(entryCard(e), lp(10)); n++; }
515         }
516         if (n == 0) {
517             TextView t = text("Keine Treffer.", 15, TXT2, false);
518             t.setGravity(Gravity.CENTER);
519             results.addView(t, lp(40));
520         }
521     }
522
523     void buildCheckTab(FrameLayout content) {
524         ScrollView sv = new ScrollView(this);
525         LinearLayout l = column(16);
526         sv.addView(l);
527         l.addView(header("Sicherheitscheck", "Lokale Analyse \u2013 nichts verlässt dein Gerät"));
528         List<Vault.Entry> weak = new ArrayList<>(), empty = new ArrayList<>(), reused = new ArrayList<>();
529         Map<String, Integer> count = new HashMap<>();
530         for (Vault.Entry e : Vault.entries) if (!e.pw.isEmpty()) {
531             Integer c = count.get(e.pw);
532             count.put(e.pw, c == null ? 1 : c + 1);
533         }
534         for (Vault.Entry e : Vault.entries) {
535             if (e.pw.isEmpty()) empty.add(e);
536             else {
537                 if (Vault.strength(e.pw) <= 1) weak.add(e);
538                 if (count.get(e.pw) > 1) reused.add(e);
539             }
540         }
541         int issues = weak.size() + empty.size() + reused.size();
542         LinearLayout sum = column(16);
543         sum.setBackground(box(SURF, 16, issues == 0 ? OK : WARN));
544         sum.addView(text(issues == 0 ? "\u2714 Keine Auffälligkeiten" : "\u26A0 " + issues + " Hinweis(e)", 18,
545                 issues == 0 ? OK : WARN, true));
546         sum.addView(text(Vault.entries.size() + " Einträge geprüft", 14, TXT2, false), lp(4));
547         l.addView(sum, lp(16));
548         addCheckGroup(l, "\u26A0 Schwache Passwörter", weak);
549         addCheckGroup(l, "\u26A0 Mehrfach verwendete Passwörter", reused);
550         addCheckGroup(l, "\u26A0 Einträge ohne Passwort", empty);
551         content.addView(sv, new FrameLayout.LayoutParams(MATCH, MATCH));
552     }
553
554     void addCheckGroup(LinearLayout l, String title, List<Vault.Entry> list) {
555         if (list.isEmpty()) return;
556         l.addView(text(title + " (" + list.size() + ")", 16, LILAC, true), lp(20));
557         for (Vault.Entry e : sorted(list)) l.addView(entryCard(e), lp(8));
558     }
559
560     void buildSettingsTab(FrameLayout content) {
561         ScrollView sv = new ScrollView(this);
```

```
        LinearLayout l = column(16);
        sv.addView(l);
        l.addView(header("Einstellungen", null));
        int secs = prefs.getInt("clip", 30);
        l.addView(text("Zwischenablage", 14, LILAC, true), lp(20));
        final Button clipBtn = btn("Automatisch löschen nach " + secs + " s", 0, TXT, null);
        clipBtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                int cur = prefs.getInt("clip", 30);
                int nxt = cur == 15 ? 30 : cur == 30 ? 60 : cur == 60 ? 120 : 15;
                prefs.edit().putInt("clip", nxt).apply();
                clipBtn.setText("Automatisch löschen nach " + nxt + " s");
            }
        });
        l.addView(clipBtn, lp(8));
        l.addView(text("Sperre", 14, LILAC, true), lp(20));
        l.addView(text("Der Tresor sperrt sich sofort, wenn du die App verlässt.", 14, TXT2, false), lp(6));
        l.addView(primaryBtn("Jetzt sperren", new View.OnClickListener() {
            @Override
            public void onClick(View v) { Vault.lock(); screen = ""; showUnlock(); }
        }), lp(8));
        l.addView(text("Konto", 14, LILAC, true), lp(20));
        l.addView(btn("Hauptpasswort ändern", 0, TXT, new View.OnClickListener() {
            @Override
            public void onClick(View v) { showChangePw(); }
        }), lp(8));
        l.addView(btn("Tresor unwiderruflich löschen", 0, DANGER, new View.OnClickListener() {
            @Override
            public void onClick(View v) { confirmWipe(); }
        }), lp(8));
        l.addView(text("Über diese App", 14, LILAC, true), lp(20));
        LinearLayout info = column(14);
        info.setBackground(box(SURF, 16, LINE));
        info.addView(text("\u2022 Verschlüsselung: AES-256-GCM\n\u2022 Schlüsselableitung: PBKDF2-HMAC-SHA256, "
                + String.format(Locale.GERMANY, "%,d", Vault.ITERATIONS) + " Iterationen\n"
                + "\u2022 Speicherung ausschließlich lokal, keine Internet-Berechtigung\n"
                + "\u2022 Kein Cloud-Backup, keine Werbung, kein Tracking\n"
                + "\u2022 Bildschirmaufnahmen sind blockiert (FLAG_SECURE)\n"
                + "\u2022 Format: eigenes verschlüsseltes Format (KFX1), noch nicht KDBX-kompatibel\n"
                + "\u2022 Version 1.0.0", 13, TXT2, false));
        l.addView(info, lp(8));
        content.addView(sv, new FrameLayout.LayoutParams(MATCH, MATCH));
    }

    void confirmWipe() {
        new AlertDialog.Builder(this, android.R.style.Theme_Material_Dialog_Alert)
                .setTitle("Tresor löschen?")
                .setMessage("Alle Einträge werden unwiderruflich gelöscht. Dies kann nicht rückgängig gemacht we
    ↳ rden.")
                .setNegativeButton("Abbrechen", null)
                .setPositiveButton("Endgültig löschen", new android.content.DialogInterface.OnClickListener() {
                    @Override
                    public void onClick(android.content.DialogInterface d, int w) {
                        Vault.wipe(MainActivity.this);
                        screen = "";
                        showSetup();
                    }
                }).show();
    }

    void showChangePw() {
        ScrollView sv = new ScrollView(this);
        LinearLayout l = column(20);
        sv.addView(l);
        l.addView(header("Hauptpasswort ändern", "Der Tresor wird mit neuem Salt neu verschlüsselt."));
        final EditText p1 = input("Neues Hauptpasswort (mind. 10 Zeichen)", true, false);
        final EditText p2 = input("Neues Hauptpasswort wiederholen", true, false);
        final TextView err = text("", 14, DANGER, false);
        l.addView(p1, lp(20));
        l.addView(p2, lp(12));
        l.addView(err, lp(8));
        final Button go = primaryBtn("Passwort ändern", null);
        go.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String a = p1.getText().toString(), b = p2.getText().toString();
                if (a.length() < 10) { err.setText("\u26A0 Mindestens 10 Zeichen erforderlich."); return; }
                if (!a.equals(b)) { err.setText("\u26A0 Die Passwörter stimmen nicht überein."); return; }
                go.setEnabled(false);
                go.setText("Wird neu verschlüsselt …");
                final char[] pw = chars(p1);
                new Thread(new Runnable() {
                    @Override
                    public void run() {
                        String problem = null;
                        try { Vault.rekey(MainActivity.this, pw); } catch (Exception ex) { problem = "Fehler: "
```

```
    ↳ + ex.getMessage(); }
648                         java.util.Arrays.fill(pw, '\0');
649                         final String pr = problem;
650                         runOnUiThread(new Runnable() {
651                             @Override
652                             public void run() {
653                                 if (pr != null) { err.setText(pr); go.setEnabled(true); go.setText("Passwort änd
    ↳ ern"); }
654                                 else { toast("Hauptpasswort geändert"); showMain(3); }
655                             }
656                         });
657                     }
658                 }).start();
659             }
660         });
661         l.addView(go, lp(8));
662         l.addView(btn("Abbrechen", 0, TXT, new View.OnClickListener() {
663             @Override
664             public void onClick(View v) { showMain(3); }
665         }), lp(8));
666         setScreen("changepw", sv);
667     }
668
669     // ---------------------------------------------------------- Editor
670
671     void showEditor(final Vault.Entry existing) {
672         final boolean isNew = existing == null;
673         final Vault.Entry e = isNew ? new Vault.Entry() : existing;
674         editing = e;
675         ScrollView sv = new ScrollView(this);
676         LinearLayout l = column(20);
677         sv.addView(l);
678         LinearLayout top = new LinearLayout(this);
679         top.setGravity(Gravity.CENTER_VERTICAL);
680         TextView back = text("\u2190", 26, TXT, true);
681         back.setPadding(dp(4), dp(4), dp(16), dp(4));
682         back.setContentDescription("Zurück");
683         back.setOnClickListener(new View.OnClickListener() {
684             @Override
685             public void onClick(View v) { showMain(tab); }
686         });
687         top.addView(back);
688         top.addView(text(isNew ? "Neuer Eintrag" : "Eintrag bearbeiten", 22, TXT, true));
689         l.addView(top);
690
691         final EditText title = input("Titel", false, false);
692         final EditText user = input("Benutzername / E-Mail", false, false);
693         final EditText url = input("URL", false, false);
694         final EditText pw = input("Passwort", true, false);
695         final EditText notes = input("Notizen", false, true);
696         title.setText(e.title);
697         user.setText(e.user);
698         url.setText(e.url);
699         pw.setText(e.pw);
700         notes.setText(e.notes);
701
702         l.addView(label("Titel"), lp(16));
703         l.addView(title, lp(4));
704         l.addView(label("Benutzername"), lp(12));
705         l.addView(user, lp(4));
706         l.addView(label("URL"), lp(12));
707         l.addView(url, lp(4));
708         l.addView(label("Passwort"), lp(12));
709         l.addView(pw, lp(4));
710         final TextView str = text("", 13, TXT2, false);
711         l.addView(str, lp(6));
712         pw.addTextChangedListener(new SimpleWatcher() {
713             @Override
714             public void afterTextChanged(Editable s) {
715                 int sc = Vault.strength(s.toString());
716                 str.setText(s.length() == 0 ? "" : "Stärke: " + strengthLabel(sc));
717                 str.setTextColor(strengthColor(sc));
718             }
719         });
720         pw.setText(e.pw);
721
722         LinearLayout pwRow = new LinearLayout(this);
723         final boolean[] shown = {false};
724         final Button show = btn("Anzeigen", 0, TXT, null);
725         show.setOnClickListener(new View.OnClickListener() {
726             @Override
727             public void onClick(View v) {
728                 shown[0] = !shown[0];
729                 pw.setInputType(InputType.TYPE_CLASS_TEXT | (shown[0]
730                         ? InputType.TYPE_TEXT_VARIATION_VISIBLE_PASSWORD : InputType.TYPE_TEXT_VARIATION_PASSWOR
    ↳ D));
731                 pw.setSelection(pw.length());
```

```
                show.setText(shown[0] ? "Verbergen" : "Anzeigen");
            }
        });
        Button gen = btn("Generieren", 0, LILAC, new View.OnClickListener() {
            @Override
            public void onClick(View v) { pw.setText(Vault.generate(20)); }
        });
        LinearLayout.LayoutParams half = new LinearLayout.LayoutParams(0, WRAP, 1f);
        half.rightMargin = dp(6);
        pwRow.addView(show, half);
        LinearLayout.LayoutParams half2 = new LinearLayout.LayoutParams(0, WRAP, 1f);
        half2.leftMargin = dp(6);
        pwRow.addView(gen, half2);
        l.addView(pwRow, lp(6));

        l.addView(label("Notizen"), lp(12));
        l.addView(notes, lp(4));

        LinearLayout copyRow = new LinearLayout(this);
        Button cu = btn("Benutzer kopieren", 0, TXT, new View.OnClickListener() {
            @Override
            public void onClick(View v) { copy("Benutzername", user.getText().toString(), false); }
        });
        Button cp = btn("Passwort kopieren", 0, TXT, new View.OnClickListener() {
            @Override
            public void onClick(View v) { copy("Passwort", pw.getText().toString(), true); }
        });
        copyRow.addView(cu, half);
        copyRow.addView(cp, half2);
        l.addView(copyRow, lp(16));

        final TextView err = text("", 14, DANGER, false);
        l.addView(err, lp(8));
        l.addView(primaryBtn("Speichern", new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String t = title.getText().toString().trim();
                if (t.isEmpty()) { err.setText("\u26A0 Bitte einen Titel angeben."); return; }
                e.title = t;
                e.user = user.getText().toString().trim();
                e.url = url.getText().toString().trim();
                e.pw = pw.getText().toString();
                e.notes = notes.getText().toString();
                e.modified = System.currentTimeMillis();
                if (isNew) Vault.entries.add(e);
                try {
                    Vault.save(MainActivity.this);
                    toast("Gespeichert");
                    showMain(tab);
                } catch (Exception ex) {
                    if (isNew) Vault.entries.remove(e);
                    err.setText("\u26A0 Speichern fehlgeschlagen: " + ex.getMessage());
                }
            }
        }), lp(4));
        if (!isNew) {
            l.addView(btn("Eintrag löschen", 0, DANGER, new View.OnClickListener() {
                @Override
                public void onClick(View v) {
                    new AlertDialog.Builder(MainActivity.this, android.R.style.Theme_Material_Dialog_Alert)
                            .setTitle("Eintrag löschen?")
                            .setMessage("„" + e.title + "“ wird gelöscht.")
                            .setNegativeButton("Abbrechen", null)
                            .setPositiveButton("Löschen", new android.content.DialogInterface.OnClickListener()
    ↳ {
                                @Override
                                public void onClick(android.content.DialogInterface d, int w) {
                                    Vault.entries.remove(e);
                                    try { Vault.save(MainActivity.this); } catch (Exception ex) { toast("Fehler:
    ↳  " + ex.getMessage()); }
                                    showMain(tab);
                                }
                            }).show();
                }
            }), lp(8));
        }
        setScreen("editor", sv);
    }

    TextView label(String s) { return text(s, 13, LILAC, true); }

    // ------------------------------------------------------------ Zwischenablage

    void copy(String what, final String value, boolean sensitive) {
        if (value.isEmpty()) { toast("Nichts zu kopieren"); return; }
        final ClipboardManager cm = (ClipboardManager) getSystemService(Context.CLIPBOARD_SERVICE);
        ClipData cd = ClipData.newPlainText("KryptoFix", value);
```

```
817         if (sensitive && Build.VERSION.SDK_INT >= 24) {
818             PersistableBundle pb = new PersistableBundle();
819             pb.putBoolean("android.content.extra.IS_SENSITIVE", true);
820             cd.getDescription().setExtras(pb);
821         }
822         cm.setPrimaryClip(cd);
823         final int secs = prefs.getInt("clip", 30);
824         toast(what + " kopiert \u2013 wird in " + secs + " s gelöscht");
825         if (clipClear != null) H.removeCallbacks(clipClear);
826         final Context app = getApplicationContext();
827         clipClear = new Runnable() {
828             @Override
829             public void run() {
830                 try {
831                     ClipboardManager m = (ClipboardManager) app.getSystemService(Context.CLIPBOARD_SERVICE);
832                     ClipData cur = m.getPrimaryClip();
833                     boolean ours = cur != null && cur.getItemCount() > 0 && cur.getItemAt(0).getText() != null
834                             && value.contentEquals(cur.getItemAt(0).getText());
835                     if (ours) {
836                         if (Build.VERSION.SDK_INT >= 28) m.clearPrimaryClip();
837                         else m.setPrimaryClip(ClipData.newPlainText("", ""));
838                     }
839                 } catch (Exception ignored) { }
840             }
841         };
842         H.postDelayed(clipClear, secs * 1000L);
843     }
844 
845     abstract static class SimpleWatcher implements TextWatcher {
846         @Override public void beforeTextChanged(CharSequence s, int a, int b, int c) { }
847         @Override public void onTextChanged(CharSequence s, int a, int b, int c) { }
848     }
849 }
```

— End of document —

KryptoFix · App created by Daniel Kiebler